

IssueScope

GitHub 仓库问答与 Issue 分析系统

前景文档

版本 1.0

需求分析负责人：焦一帆

2026 年 7 月

项目名称	IssueScope（GitHub 仓库问答与 Issue 分析系统）
文档名称	前景文档
版本号	1.0
制定日期	2026-07-07
制定人	焦一帆
评审日期	2026-07-09

修订历史记录

版本	日期	修订内容	修订人	审核人
1.0	2026-07-07	初稿	焦一帆	全体成员（评审 日期 07-09）

目录

1 简介

1.1 目的

本文档旨在定义 IssueScope（GitHub 仓库问答与 Issue 分析系统）的高层次需求，阐述搭建一个将 GitHub 公开仓库自动同步、文件与 Issue 分类、智能问答和分析的平台的核心目标与范围，并规划引入 RAG、LLM、多 Agent 等进阶能力的远景方向。

1.2 范围

本系统是一个面向 GitHub 仓库的全链路智能分析平台，涵盖以下功能范围：

- **仓库接入与同步**——支持公开 GitHub 仓库接入（REST API），自动同步元数据、README、目录树、Issues、PRs、Commits 数据
- **知识库构建**——文件自动分类、知识图谱与向量索引相结合的仓库知识库
- **智能问答**——基于仓库上下文的 What / Where / How / Why 问答
- **Issue 智能分析**——9 类自动分类、重复检测、置信度评分，通过 Webhook 实时监听
- **自动回复与修复**——生成回复建议，定位相关代码模块并自动创建修复 PR
- **知识沉淀**——历史修复模式学习、维护者偏好适配与 FAQ 知识库自动生成
- **可视化仪表盘**——仓库全景信息展示与数据分析

1.3 定义、首字母缩写词和缩略语

术语	定义
Agent	具有自主决策和行动能力的智能体
RAG	Retrieval-Augmented Generation，检索增强生成
LLM	Large Language Model，大语言模型
Issue	GitHub 项目中的问题/任务条目
PR	Pull Request，代码合并请求
KB	Knowledge Base，知识库
FAQ	Frequently Asked Questions，常见问题
RepositorySnapshot	仓库同步后生成的全量数据快照，包含元信息、文件分类、Issue 分类等
FileCategory	文件分类体系：SOURCE / TEST / DOCS / CONFIG / CI_CD / DEPENDENCY / BUILD / ASSET / DATA / OTHER
IssueCategory	Issue 分类体系：BUG / FEATURE_REQUEST / QUESTION / DOCUMENTATION / DUPLICATE / INFO_NEEDED / INVALID / MAINTENANCE / UNKNOWN
Webhook	GitHub 事件推送机制，支持 issues.opened 事件的接收和分类
Sync	仓库同步——拉取 GitHub 数据 → 规则分类 → 生成快照 → 缓存

1.4 参考资料

1. GitHub REST API 文档, <https://docs.github.com/en/rest>
2. GitHub Webhook 文档, <https://docs.github.com/en/webhooks>

2 定位

2.1 商机

随着开源生态的快速发展，GitHub 上仅有少部分项目配置了 Issue 模板，大多数项目的 Issue 处于无分类管理状态。项目维护者往往需要花费大量时间手动分类和回复 Issue。

现有的 GitHub Issues 原生工具仅有标签和里程碑功能，缺乏智能分析和回复能力。当前市场上缺少能够将仓库代码理解、Issue 智能分类、自动问答有机结合的一体化解决方案，这为本系统提供了明确的商机。

2.2 问题说明

面对问题	Issue 类型混杂，维护者需手动分类
影响	开源项目维护者、核心贡献者、普通开发者
后果	响应延迟，关键 Bug 被淹没，社区满意度下降，贡献者流失
成功方案	自动 Issue 分类（规则分类与 LLM 增强分类）

面对问题	重复问题反复出现
影响	开源项目维护者
后果	维护者反复回复相同内容，维护效率低下
成功方案	重复检测 + FAQ 自动生成

面对问题	新贡献者不熟悉代码库结构和设计
影响	新贡献者和普通开发者
后果	提问质量低，交流成本高，社区入门门槛高
成功方案	仓库知识库智能问答（What / Where / How）

面对问题	Bug 修复定位困难
影响	项目维护者和核心贡献者
后果	修复周期长，Issue 积压
成功方案	自动定位相关文件 + 修复建议

面对问题	缺乏 Issue 处理后的知识沉淀
影响	开源项目社区
后果	相同问题反复被问，社区知识无法积累
成功方案	自动生成 FAQ 知识库

2.3 产品定位说明

针对于	开源项目维护者和核心贡献者
他们是	需要高效管理 GitHub Issue、快速响应社区的开发者和项目团队
我们的产品	IssueScope（GitHub 仓库问答与 Issue 分析系统）
功能是	自动同步分析 GitHub 仓库，智能分类 Issue，提供基于代码上下文的问答和修复建议
不同于	GitHub Issues 原生工具（仅标签分类）和通用 AI 编码助手（无仓库上下文）
我们的产品	专注于仓库级别的上下文理解，将代码结构、Issue 历史、文档知识整合为统一知识库

3 涉众和用户说明

3.1 市场统计

目标用户群体包括：

- 开源项目维护者——拥有仓库管理权限，负责处理 Issue
- 核心贡献者——经常参与项目，了解代码结构
- 普通开发者——偶尔参与项目的贡献者或用户

本产品不区分个人/企业付费模式，不涉及商业模式。随着 AI 与 DevOps 的深度融合，智能化的代码仓库管理工具将成为开发者工具链中的重要组成部分，市场需求呈现快速增长趋势。

3.2 涉众概要

涉众	说明
产品经理 / 需求分析师	定义产品方向、管理需求、评审文档
开发团队（前端 + 后端 + AI）	实现系统功能、维护代码质量
测试 / QA 团队	验证功能和性能、保证质量
开源社区 / 最终用户	实际使用系统的开源维护者和开发者

3.3 用户概要

用户类型	描述	核心需求
项目维护者	拥有仓库管理权限，负责处理 Issue	Issue 自动分类、回复建议、自动修复
核心贡献者	经常参与项目，了解代码结构	代码问答、Issue 定位协助
普通开发者	偶尔参与项目的贡献者或用户	仓库知识问答、使用问题咨询

产品不区分个人/企业付费用户，不涉商业模式。

3.4 关键涉众/用户需要

需求	优先级	说明
GitHub 仓库接入	高	系统基础能力，同步元数据、文件、Issues 等
仓库知识库构建	高	文件自动分类，知识图谱与向量索引
仓库智能问答	高	基于仓库上下文的 What/Where/How 问答
Issue 智能分析	高	自动分类、重复检测、置信度评分
Web 浏览器客户端	高	响应式前端界面
自动回复草稿生成	中	为常见 Issue 类型生成回复建议
响应式界面	中	适配不同屏幕尺寸
Issue 自动修复与 PR 创建	中	自动定位、修改代码并提交 PR
多项目并发处理	低	同时管理多个 GitHub 仓库
长期记忆和偏好学习	低	学习维护者偏好与历史修复模式
FAQ 自动生成	低	从历史 Issue 提取高频问题并生成 FAQ

3.5 备选方案和竞争

竞品	与我们的差异
GitHub Copilot / Copilot Chat	代码补全和通用代码问答，无仓库级别上下文，不聚焦 Issue 管理
GitHub Issues 原生工具	标签 + 里程碑分类，仅能手动分类，无智能分析和自动回复
Sourcegraph	代码搜索和导航平台，侧重代码浏览而非 Issue 管理和分析
Jira	企业级项目管理，重量级，不适合开源社区场景
Sweep AI	最直接竞品，自动将 Issue 转为 PR；我们覆盖更多（问答 + FAQ + 分析）
CodeRabbit	AI PR 审查，从 PR 阶段切入；我们覆盖从 Issue 到修复的全流程

4 产品概述

4.1 产品总体效果

本系统作为连接 GitHub 仓库与智能分析的中间层平台，通过以下核心流程工作：

1. 接入 GitHub 仓库，自动同步代码、Issue、PR、文档等资源
2. 构建项目知识库，包含文件分类、知识图谱与向量索引
3. 提供智能问答接口，回答仓库相关问题
4. 自动监听并分析新 Issue，进行分类与建议
5. 自动生成修复方案并发起 PR

4.2 功能摘要

用户利益	支持特性
快速获取仓库全景信息	GitHub 仓库一键同步 + 仪表盘展示
了解仓库文件构成	文件自动分类（10 类）+ 分类统计
自动识别 Issue 类型	Issue 规则分类（9 类）+ 置信度评分
获得 Issue 处理建议	分类结果附带建议操作文本
实时获取新 Issue 通知	Webhook 监听 + 自动分类入库

4.3 假设与依赖关系

- 依赖 GitHub REST API 的可用性和速率限制
- 依赖大语言模型服务的可用性与响应性能
- **LLM 服务费用**：每次问答和分类调用会产生 token 费用
- **GitHub API 变更**：依赖 GitHub REST API，API 版本变更可能影响功能
- 假设目标仓库具有合理的目录结构和代码组织
- 需要足够的存储资源以支持知识库构建

5 产品特性

5.1 仓库接入与管理

- **认证方式**：支持 Personal Access Token（通过 GITHUB_TOKEN 环境变量配置），不支持 OAuth 或 GitHub App
- **接入方式**：用户输入公开 GitHub 仓库 URL，系统使用 REST API 拉取数据

- 同步内容：仓库元数据、README、目录树、Issues、PRs、Commits
- 组织级批量接入：一次接入一个指定 URL

5.2 知识库构建

- 文件分类：FileClassifier 按路径规则判断文件类别，共 10 类
- 知识图谱技术：待评估 Neo4j 或自建图结构
- 向量数据库：pgvector（与 PostgreSQL 统一）或 Chroma
- 代码分析粒度：文件级分类，可细化到函数/类级

5.3 智能问答

支持以下类型的问题回答：

- **What 型**：询问某个模块的功能、某个文件的作用等
- **Where 型**：查找特定功能在代码中的位置
- **How 型**：如何使用某个 API、如何配置环境等
- **Why 型**：分析 Issue 产生原因或设计决策理由

5.4 Issue 智能分析

系统自动监听新提交的 Issue，进行多维度分析：

- 类型识别：覆盖 9 类——BUG / FEATURE_REQUEST / QUESTION / DOCUMENTATION / DUPLICATE / INFO_NEEDED / INVALID / MAINTENANCE / UNKNOWN
- 自动回复：分类结果附带 suggested_action 建议文本，由维护者决定是否采纳（需要人工审核后发布）
- 置信度评分：每条分类结果附带置信度分数
- 实时监听：通过 Webhook 接收 issues.opened 事件，自动分类入库

5.5 Issue 自动修复与变更生成（进阶）

对于需要修改代码的 Issue：

- 结合 Issue 描述、项目上下文和历史记录定位相关模块与文件
- 生成修复方案并自动修改代码
- 创建新分支、提交修改并发起 Pull Request（允许维护者审阅后再合并）
- 测试验证：配合测试流程验证修复正确性

5.6 长期记忆与学习（进阶）

记录和分析项目的：

- 维护者的 Review 偏好
- 代码风格偏好
- 历史修复模式的向量化存储
- 后续 Issue 处理中自动应用积累的经验

5.7 FAQ 知识库（进阶）

- 自动从历史 Issue 中提取高频问题，生成结构化的 FAQ 知识库
- 发布方式：系统自动生成 + 人工审核后发布
- 更新频率：定期更新或按 Issue 量触发

6 约束

- 系统需兼容 GitHub API v3/v4（REST + GraphQL）

- 需遵守 GitHub 平台的 API 使用限制和速率限制政策
- **LLM API 成本约束**：调用外部 LLM 服务有 token 费用
- **公开仓库优先**：支持公开仓库，私有仓库需额外 OAuth 认证流程
- 前端需适配主流浏览器（Chrome、Firefox、Safari、Edge）
- 需保证用户数据安全（特别是私有仓库的代码数据）
- **无合规性约束**：课程项目，不涉及合规要求

7 质量属性

质量属性	目标值
Issue 分类准确率（规则版）	> 80%
Issue 分类准确率（LLM 增强）	> 90%
问答准确率	> 80%
仓库同步成功率	> 95%
Webhook 事件处理延迟	< 5 秒
前端页面加载时间	< 3 秒
系统可用性	全天可用

8 优先级

优先级	范围
P1（核心）	仓库接入、知识库构建（文件分类）、智能问答、Issue 分析、Web 客户端
P2（进阶）	自动回复草稿、响应式界面、多项目并发
P3（增强）	Issue 自动修复与 PR、长期记忆、FAQ 生成

9 其他产品需求

9.1 适用的标准

- **安全标准**：Personal Access Token 认证与 OAuth 2.0
- **通信标准**：HTTPS/TLS 加密传输，RESTful API 设计规范
- **代码标准**：遵循 Semantic Versioning 语义化版本规范
- **数据标准**：JSON 作为主要数据交换格式

9.2 系统需求

模块	选型结果
后端语言	Python
Web 框架	FastAPI
LLM 服务	DeepSeek API
关系数据库	PostgreSQL + pgvector
图数据库	待评估（Neo4j 或自建方案）
向量数据库	pgvector（与 PostgreSQL 统一，降低部署复杂度）
任务队列	Celery + Redis
部署方式	Docker Compose
版本管理	Git + GitHub
RAG 框架	LlamaIndex 或轻量自研
Agent 框架	Openai SDK 或自研任务流

9.3 前端技术选型

模块	选型结果
前端框架	React + Vite + TypeScript
UI 组件库	lucide-react 图标 + Shadcn/ui（基于 Tailwind，渐进式引入）
CSS 方案	Tailwind CSS

9.4 环境需求

- 前端为浏览器端应用，不依赖客户端安装
- 后端可部署于云服务器或私有服务器
- 需要稳定的网络连接以访问 GitHub API 和 LLM 服务
- 提供 Docker Compose 一键部署方案（后端 + 前端 + 数据库 + 任务队列统一编排）

10 文档需求

10.1 用户手册

提供以下内容的用户操作指南：

- Web 界面操作指南（仓库接入、查看分析、提问、Issue 处理）
- API 开发者文档（FastAPI 自动生成 Swagger UI）

由于系统是 API 驱动的，API 文档重要性高。API 框架内置 Swagger UI 自动生成。

10.2 联机帮助

- 悬浮 tooltip（表单字段提示、图表说明）
- 引导教程

10.3 安装指南与自述文件

提供系统部署文档，包括：

- 环境要求
- 安装步骤（Docker Compose 一键部署方案）

- 配置文件说明
- 常见问题解答